A better school timetable

We built a program that makes the timetable for 626 students, 54 teachers and 63 rooms. Students walk about half as much, free periods stop piling up between lessons, a separate program checks every rule, and the plan still works when a room closes or a teacher is away. The timetable is the same every week.

−56% less walking for students each week

−93% fewer free periods stuck between lessons

0 broken rules out of 10, checked by a separate program

10/10 deliberate mistakes the checker caught

A What we were asked to do

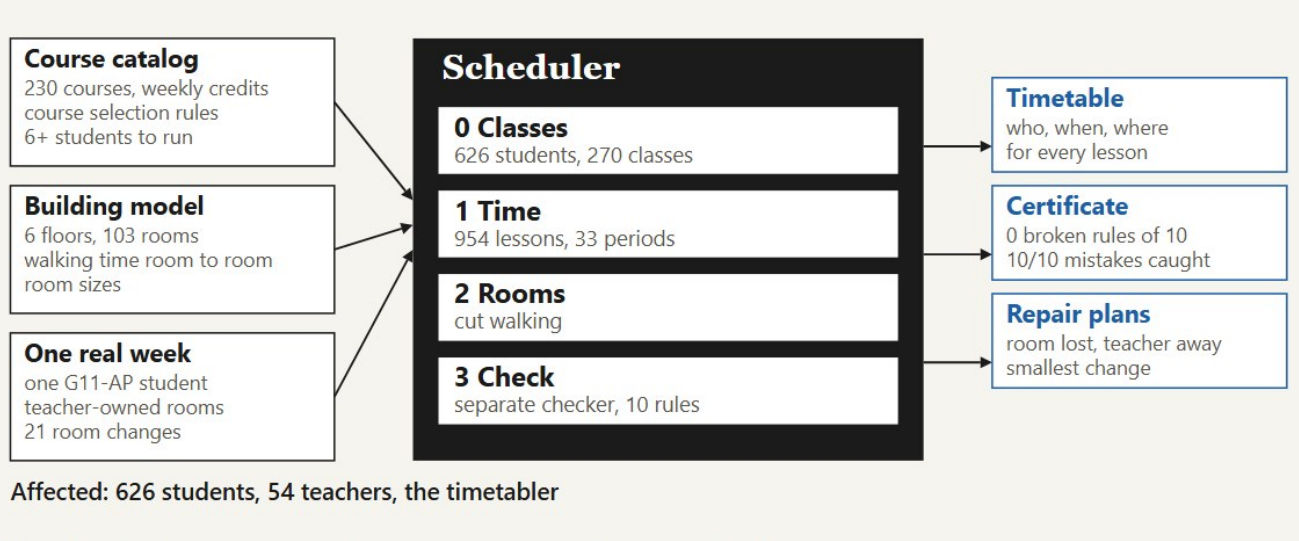
a school of 626 students, 54 teachers and 63 rooms

The brief asks for three things: make a school timetable automatically, show that it is better than the current one, and prove that it follows every rule. We were given the course catalog, a model of the building and one real student's week. Around those we built a simulated school, wrote a scheduler, and compared it with a stand-in for how timetables are made today. **Every week uses the same timetable**, so we plan one week and it repeats.

THE BRIEF · Question.txt, verbatim

> The "schoo" has to produce a timetable that takes care of teachers, students, classrooms and courses all at once. This timetable affects the daily study and work of teachers and students, so having a high-quality timetable is important. We want you to consider the constraints in the school's timetable and optimize the timetable. By this, you should design a system and be able to explain why it is better than what we currently have. An example of a constraint is class hours: you may change Math class with English class, but the hours spent on math class in a semester should not change. The result should be a system that automatically schedules timetables. There is no standard answer. How the current system works, which constraints must be satisfied, which aspects are worth improving: all of these are for you to research, judge and trade off, and you should explain your reasoning in your submission. During the presentation, the judges will ask why you made these decisions.
>
> **2. Evaluating Your Results**
>
> This problem cares more about which aspects you think can be improved and how you improve them. Your submission should answer:
>
> - What problems in the current timetable system did you identify?
> - Which aspects did you choose to optimize (for example, minimizing student movement between classrooms)? How did you optimize them?
> - Who do these aspects affect, and how much? Why do they really matter?
> - Compared with the current timetable, how much does your solution improve things? How do you measure it?
> - How do you confirm that your timetable satisfies every constraint it must satisfy?
> - Can your system still cope when circumstances change? How to present your results most convincingly is also up to you.

Q1	What is wrong with today's timetable? → **section B**
Q2	What did we improve, and how? → **C, D**
Q3	Who is affected, and how much? → **D**
Q4	How much better is it, and how is that measured? → **D**
Q5	How do we know every rule is followed? → **E**
Q6	Does it cope when things change? → **F, G**



Course catalog 230 courses, weekly credits; course selection rules; 6+ students to run

Building model 6 floors, 103 rooms; walking time room to room; room sizes

One real week one G11-AP student; teacher-owned rooms; 21 room changes

Scheduler
- **0 Classes** 626 students, 270 classes
- **1 Time** 954 lessons, 33 periods
- **2 Rooms** cut walking
- **3 Check** separate checker, 10 rules

Timetable who, when, where for every lesson

Certificate 0 broken rules of 10; 10/10 mistakes caught

Repair plans room lost, teacher away; smallest change

Affected: 626 students, 54 teachers, the timetabler

Fig. 1 — *What goes in, what the program does, and what comes out.*

What is real, what is simulated

Real: the course catalog and its selection rules, the six-floor building with the walking time between any two rooms, and one student's observed week.

Simulated: the 626 students and their course choices, the 54 teachers, and the "current method" we compare against. Everything is generated from a fixed seed, so anyone can re-run it and get the same numbers.

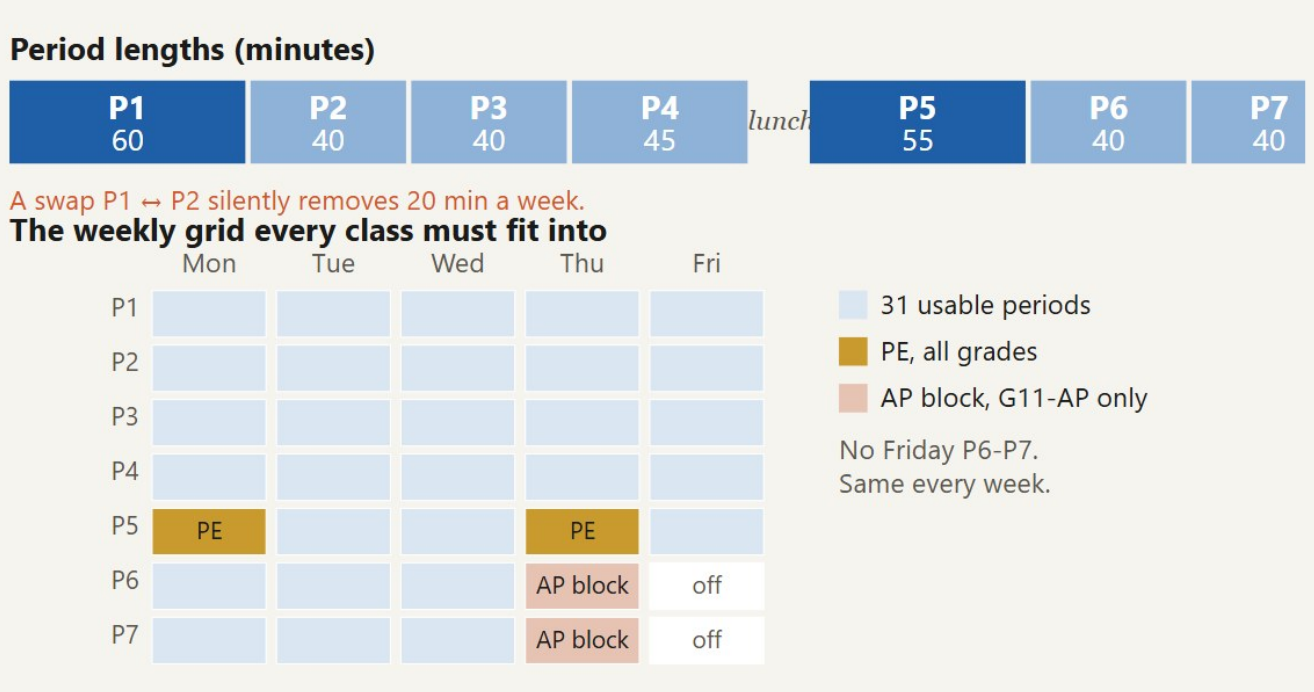


Fig. 2 — *The 33 periods a class can use in a week, and how long each one is.*

Why we count minutes, not lessons

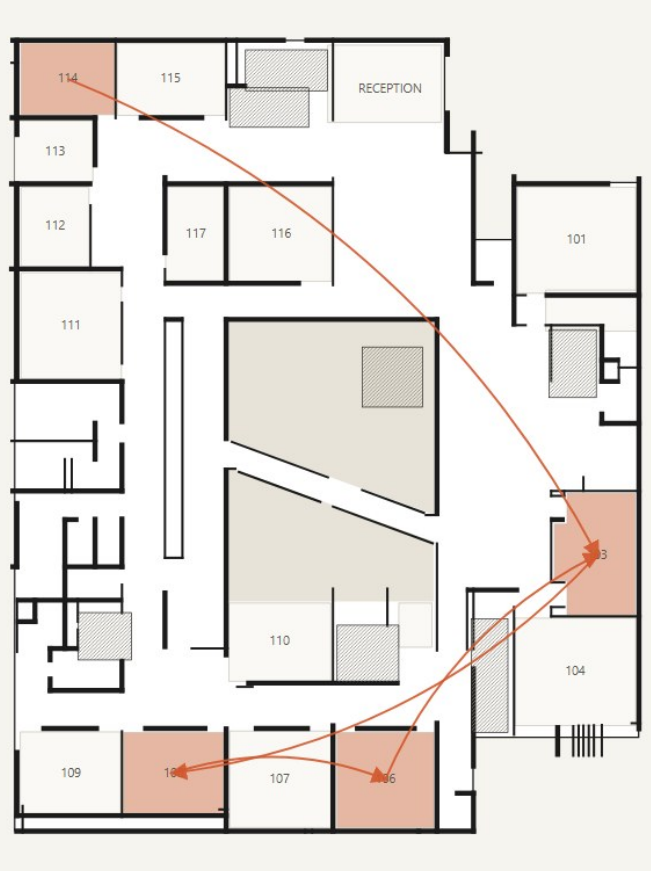
A lesson in period 1 lasts 60 minutes, in period 2 only 40. Moving a Math lesson from one to the other keeps the lesson count but quietly removes 20 minutes. The brief says class hours must not change, so we count minutes.

Some periods are not available: PE takes Monday and Thursday period 5, and there is no Friday afternoon. This one grid repeats every week.

B What is wrong with the current timetable

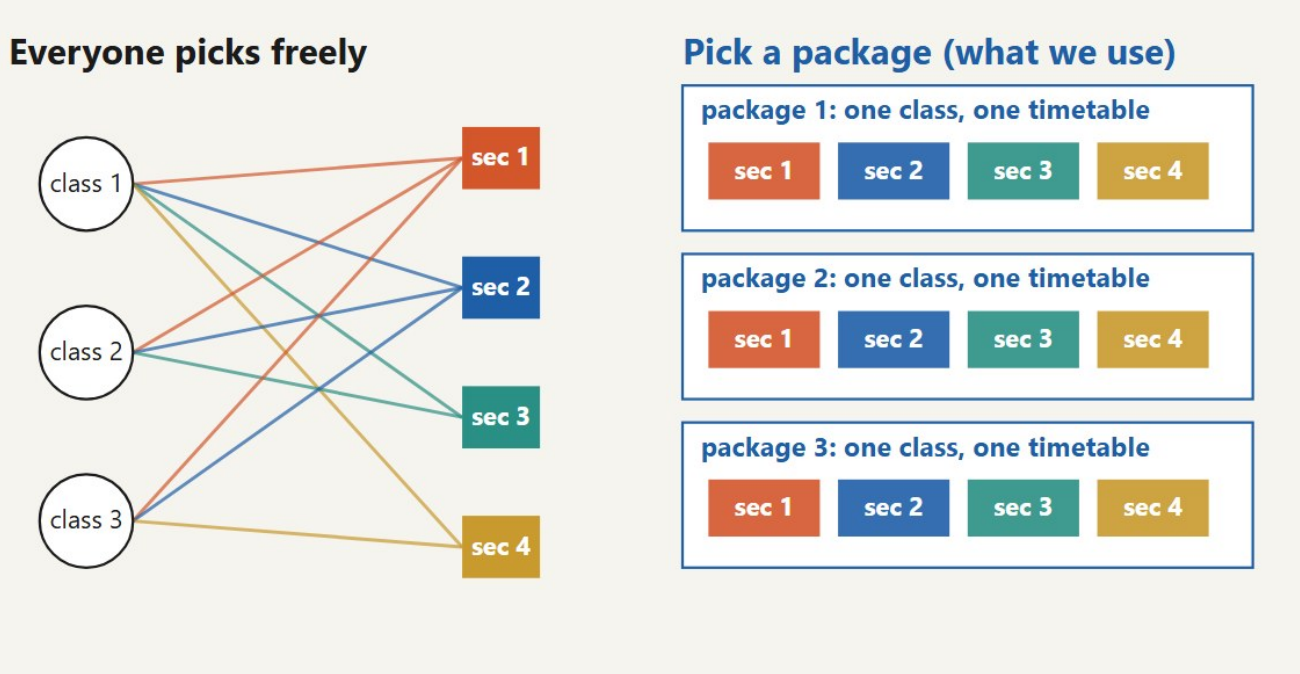
Question 1

In the one real week we have, a student changes room 21 times, because every teacher keeps their own room and students walk to them. Free periods end up scattered between lessons, and nobody checks that a lesson keeps the same number of minutes after it is moved. The three figures below show each problem in turn.



Monday of the one real week: 114 → 103 → 108 → 108 → 106 → 106 → 103
4 room changes in one day; **21** per week, 488 s of walking.

Fig. 3 — *One student's real Monday on the ground-floor plan: each arrow is a room change. Every teacher owns a room, so the student walks to the teacher all day.*



Everyone picks freely
class 1
class 2
class 3
sec 1
sec 2
sec 3
sec 4
Pick a package (what we use)
package 1: one class, one timetable
sec 1
sec 2
sec 3
sec 4
package 2: one class, one timetable
sec 1
sec 2
sec 3
sec 4
package 3: one class, one timetable
sec 1
sec 2
sec 3
sec 4

Sections tie classes together; lessons that may not overlap **form a clique of 31 > 29 free periods: no valid grid.**

Each section holds one class: cliques stay below **the slot count; conflicts shrink to teachers, rooms.**

Fig. 4 — *Left: free choice. Right: students pick a package.*

Why free choice breaks the timetable

If students can mix electives any way they like, each class shares students with many other classes, and all of those need different time slots. In our school that means 31 lessons that must not overlap, but only 29 free periods: no valid timetable exists. So we offer a limited set of course packages. Students in one package form a class that takes all its lessons together.

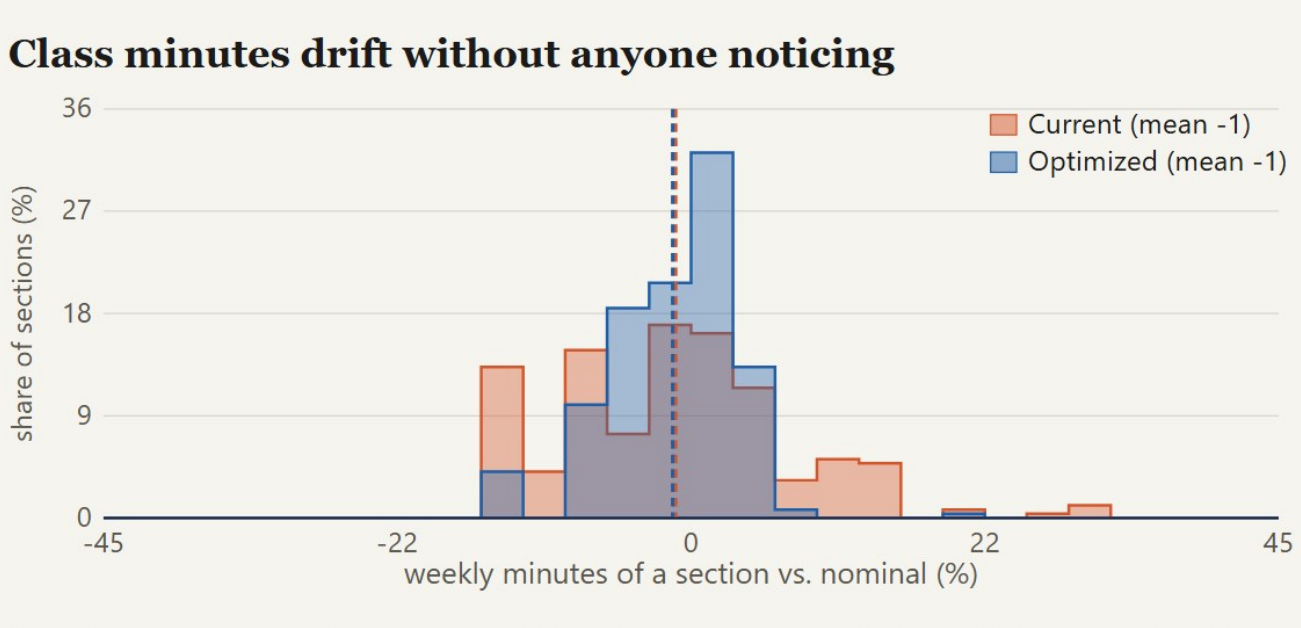


Fig. 5 — *Each class's weekly minutes compared with the normal amount. A timetable built without looking at period length puts* **110 of 270** *classes outside the ±8% allowed band; ours puts none (the brief: class hours should not change).*

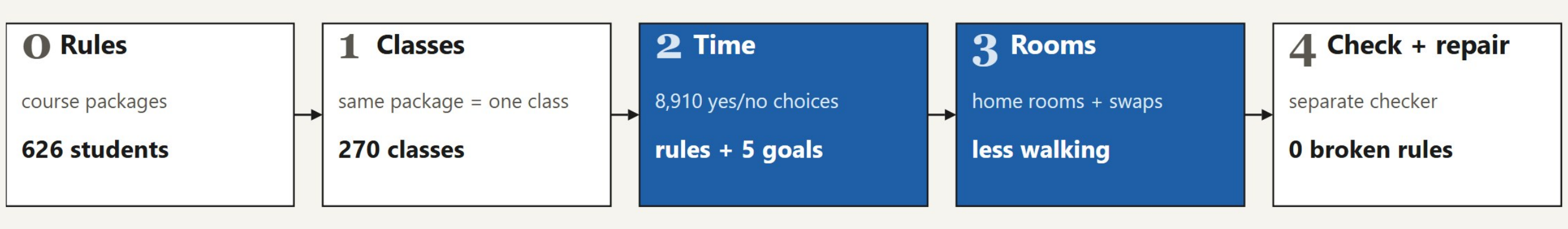
- Nothing aims to reduce students' walking, free periods stuck between lessons, or overloaded days.
- When a room is lost, the fix is made by hand.
- There is no independent proof that the timetable follows the rules.

In short: lots of walking, wasted gaps, and no check on class hours.

C How we built it

Question 2

We split the job into steps that can each be checked on their own. First, group students into classes. Second, choose when each class meets. Third, choose the room. Fourth, check every rule on the finished timetable and keep a quick way to repair it. The equations at the bottom are the exact rules and goals the computer works with.



0 Rules course packages **626 students**

1 Classes same package = one class **270 classes**

2 Time 8,910 yes/no choices **rules + 5 goals**

3 Rooms home rooms + swaps **less walking**

4 Check + repair separate checker **0 broken rules**

Fig. 6 — *The five steps, with the size of the problem at each one.*

0 Rules
Course credits become weekly lessons. The selection rules decide which course packages are allowed.

1 Classes
Students with the same package form a class of at most 24. A course needs at least 6 students to run.

2 Time
The solver picks the period for every lesson, obeying all hard rules and cutting idle gaps. It uses one yes/no variable per class and period, so it stays fast.

3 Rooms
Each class gets a home room. Leftover lessons are matched to free rooms, then swapped to cut walking.

4 Check
A separate program checks the ten rules. If something changes, only the affected lessons are re-planned.

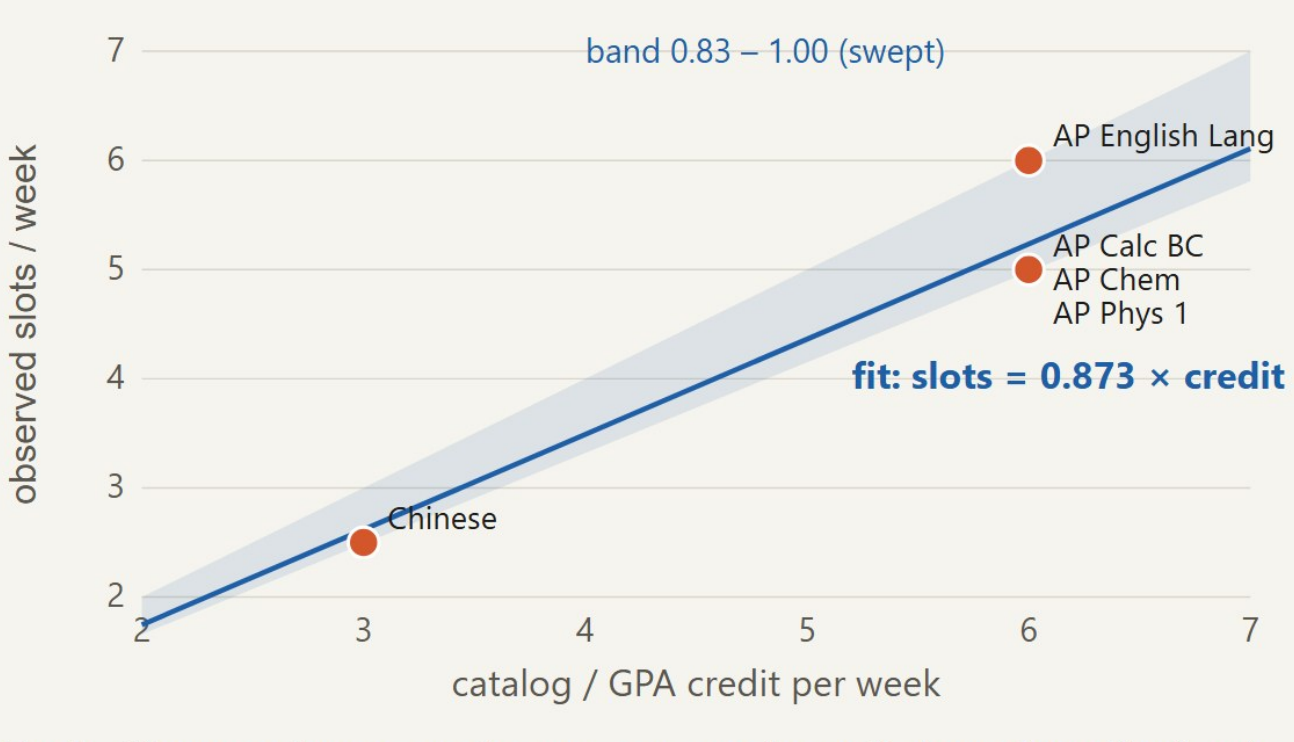


Fig. 7 — *How many lessons a week a course gets: catalog credit times a factor fitted on the real week, tried across a whole band of values.*

CREDIT TO WEEKLY SLOTS

$$n_c = \left\lfloor k\kappa_c + \tfrac{1}{2} \right\rfloor, \quad k = \frac{\sum_i \kappa_i o_i}{\sum_i \kappa_i^2} \approx 0.873$$

HARD RULES ($z_{g,s} \in \{0,1\}$)

$$\sum_s z_{g,s} = n_g, \quad \sum_{g \in \sigma} z_{g,s} \le 1, \quad \sum_{g \in T_t} z_{g,s} \le 1$$

$$\sum_{g:\, a_g \ge a} z_{g,s} \le R_\kappa(a) \quad \text{(room area)}$$

CONTACT HOURS IN MINUTES

$$\left| \sum_s \mu_s z_{g,s} - n_g \bar{\mu} \right| \le \max(\tau n_g \bar{\mu},\, 9), \ \tau = 0.08$$

STAGE 1: SLOT OBJECTIVE

$$\min\ w_{gap} \sum_\sigma c_\sigma \Gamma_\sigma + w_h \sum_\sigma c_\sigma H_\sigma + w_{sp} \sum_g |g|\, \mathrm{rep}_g + w_m \sum_{g \text{ hard}} |g| \sum_s (p_s - 1) z_{g,s} + w_{tg} \sum_t \Gamma_t$$

STAGE 2: PERSON-SECONDS

$$\min_r \sum_{(a,b) \in \mathcal{L}_{stud}} n_{ab}\, \tau(r_a, r_b) + \lambda \sum_{(a,b) \in \mathcal{L}_{teach}} \tau(r_a, r_b)$$

SCORE, FAIRNESS, ROBUSTNESS

$$J = \bar{\ell}_{stud} + \alpha \bar{L}_t + \beta\, \mathrm{CVaR}_q(L), \qquad \mathbb{E}[\Delta] = \sum_\xi p_\xi \Delta(\xi)$$

Students stay, teachers walk: λ says how much one second of teacher walking counts against one second of student walking. Γ = free periods stuck between lessons, H = days with six or more classes, rep = a class meeting twice on one day.

In plain words: every class gets exactly its number of lessons; nobody (student, teacher or room) is in two places at once; weekly minutes stay near the normal amount; then the computer removes as many wasted gaps and as much walking as it can.

D How much better is it?

Questions 3 and 4

To measure the gain we built 24 timetables the way the school makes them today (any valid timetable, teacher-owned rooms) and compared them with ours on the same school. In every chart a lower number is better. The floor plans show where students change rooms, the charts show what that means for each student, and the teacher grids show who ends up with free periods stuck between lessons.

🟥 Current method (24 simulated timetables, tuned to match the real week; the plots show run 0) 🟦 Optimized

Where students walk

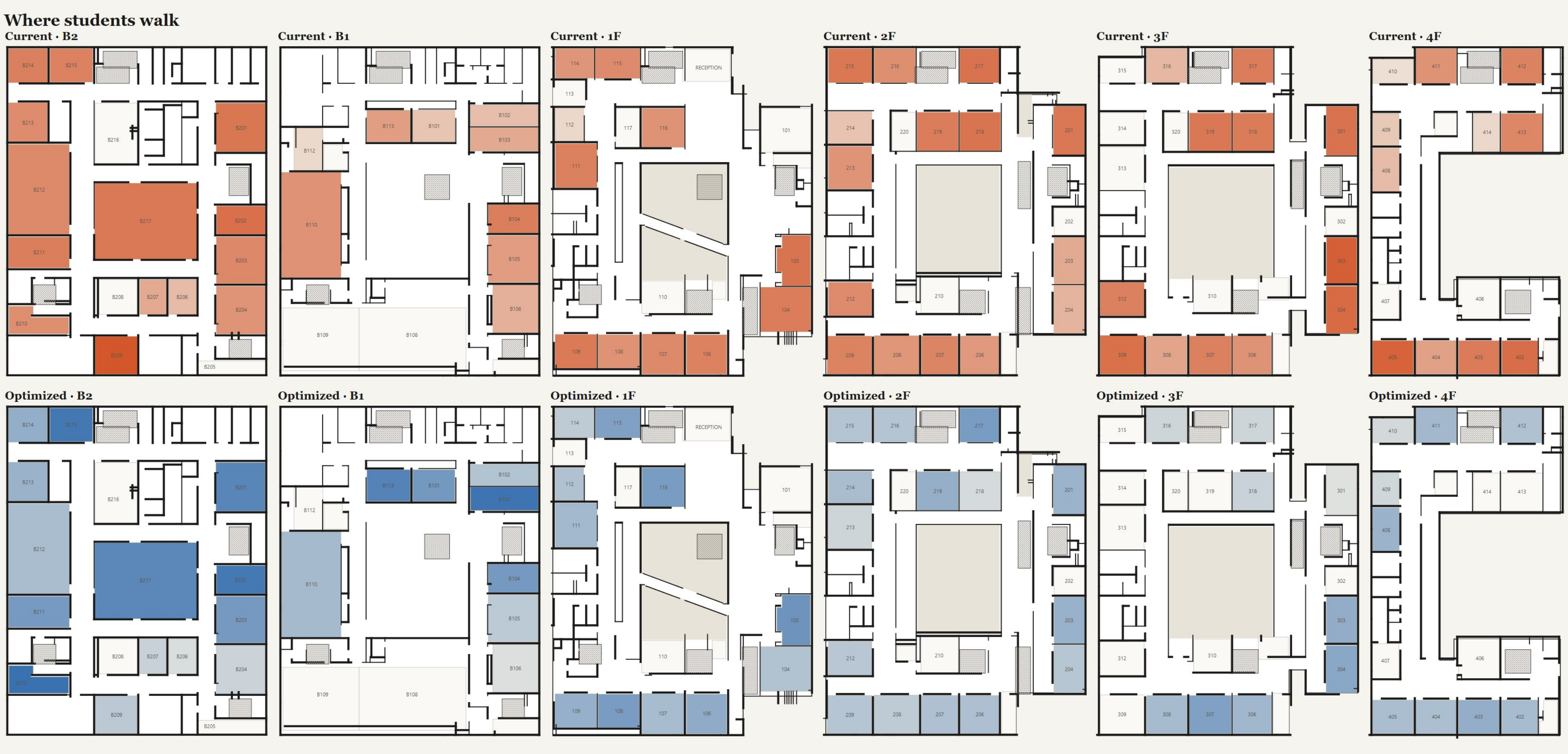


Fig. 8 — *Real building, six floors, drawn to scale. The darker the room, the more room changes touch it each week (same scale for both rows). With teacher-owned rooms, walking spreads across the whole building; with home-class rooms it stays near where each class lives.*

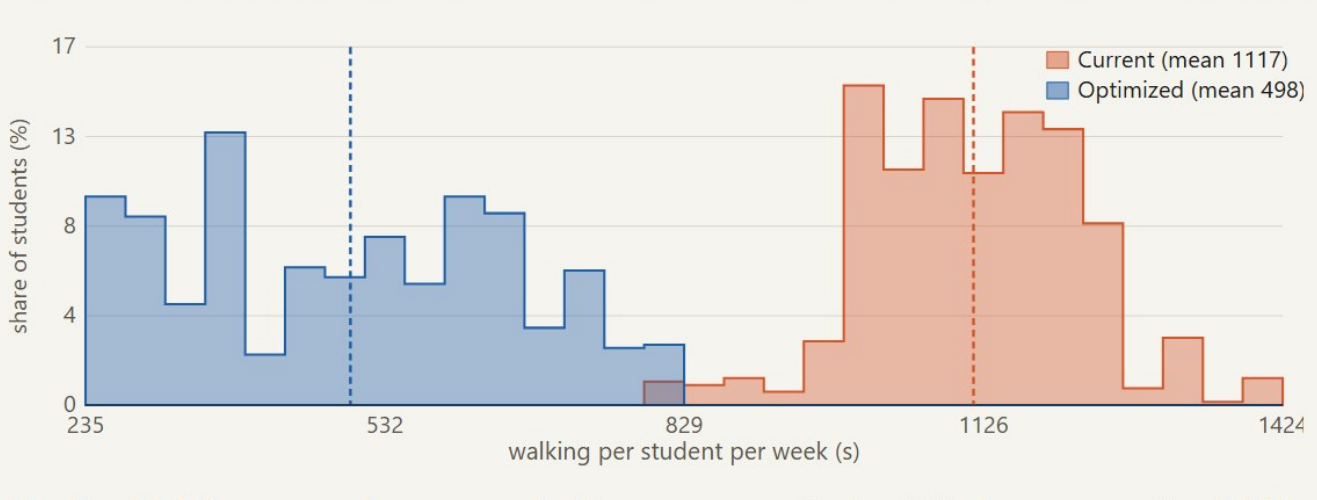


Fig. 9 — *Walking per student per week. Mean 1122 → 498 s (−56%); the worst-20% tail falls too, so the gain does not come from a few students.*

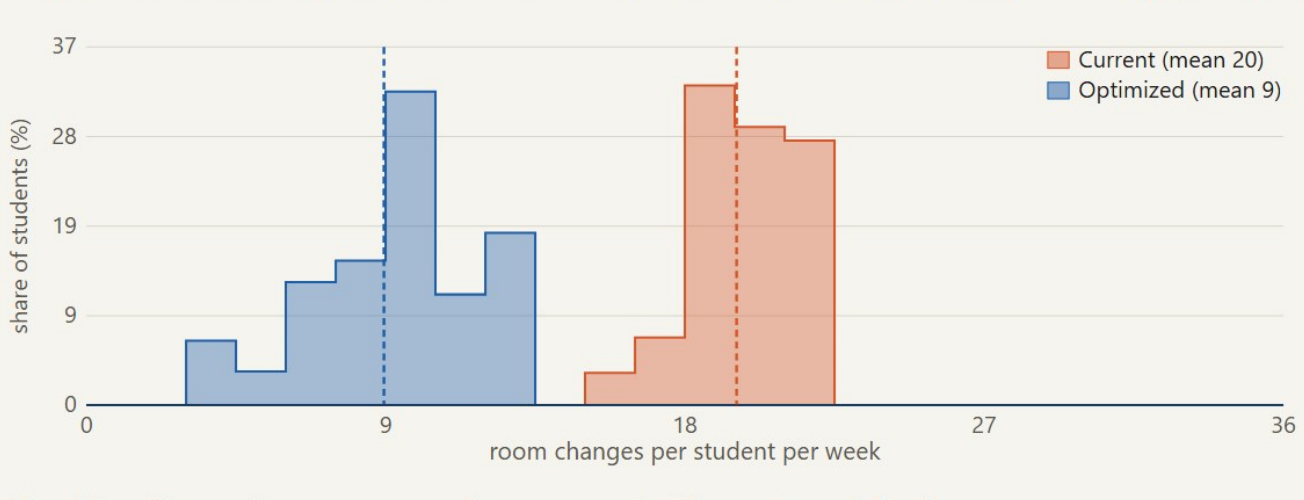


Fig. 10 — *Room changes per student per week. The real week had 21.*

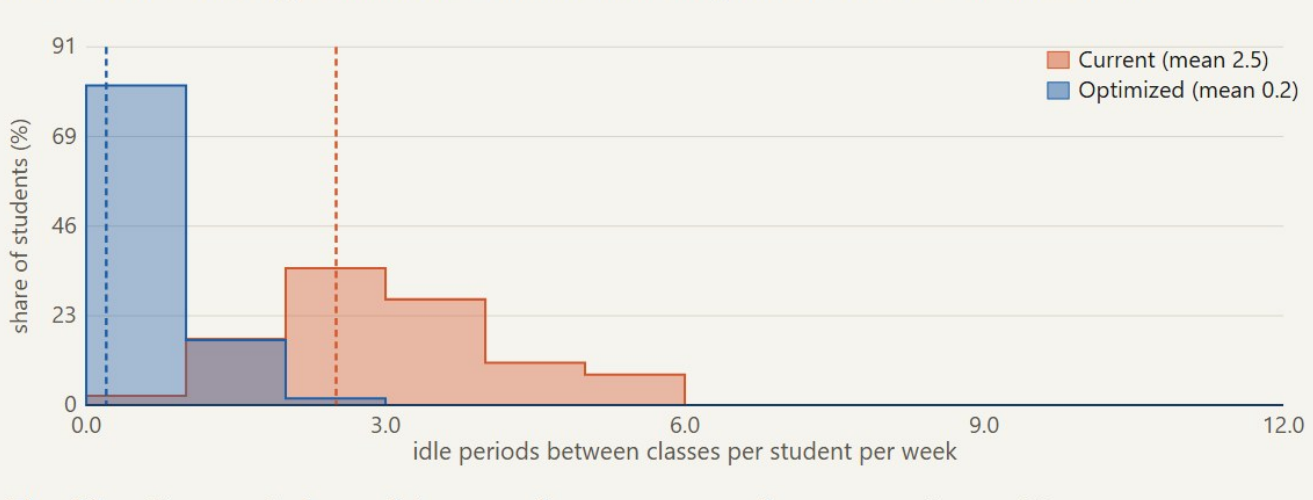


Fig. 11 — *Free periods stuck between lessons, per student per week: −93%.*

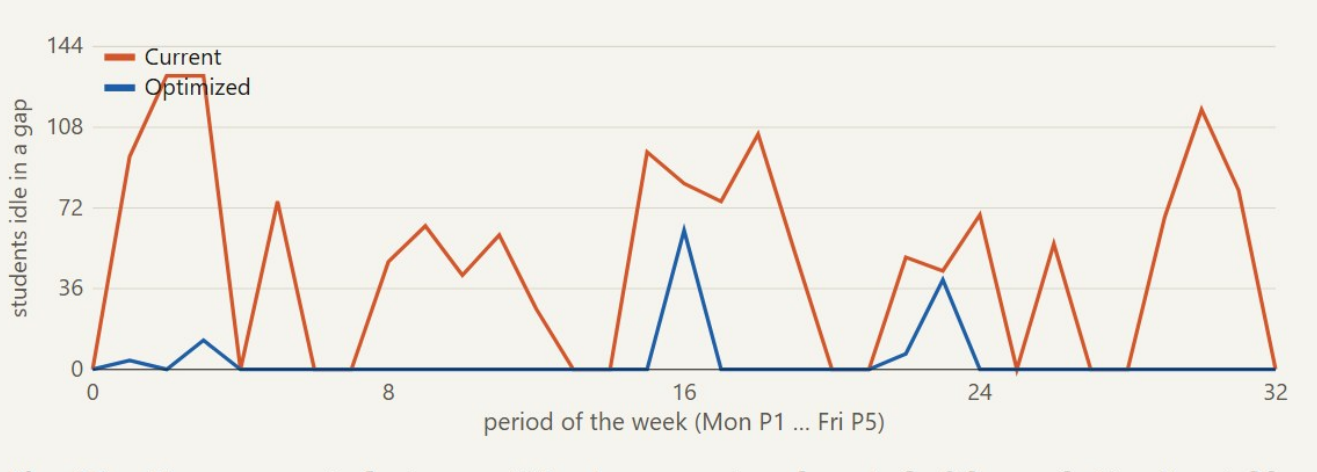


Fig. 12 — *How many students are sitting in a gap at each period of the week. Our timetable moves free periods to the start or end of the day.*

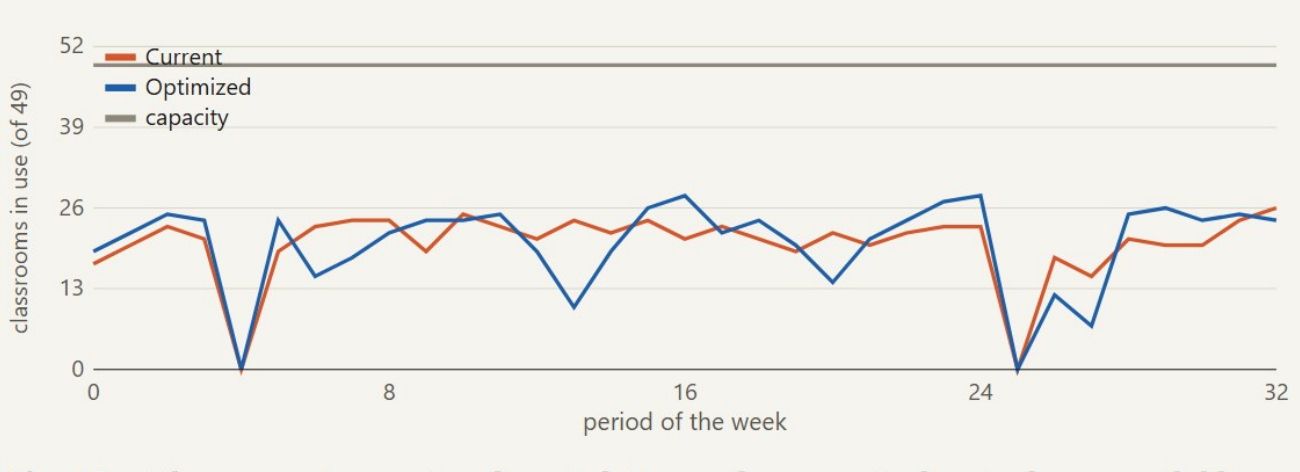


Fig. 13 — *Classrooms in use at each period. Demand never gets close to the 49 available, so the saving does not come from squeezing into fewer rooms.*

Which change saves the most walking?

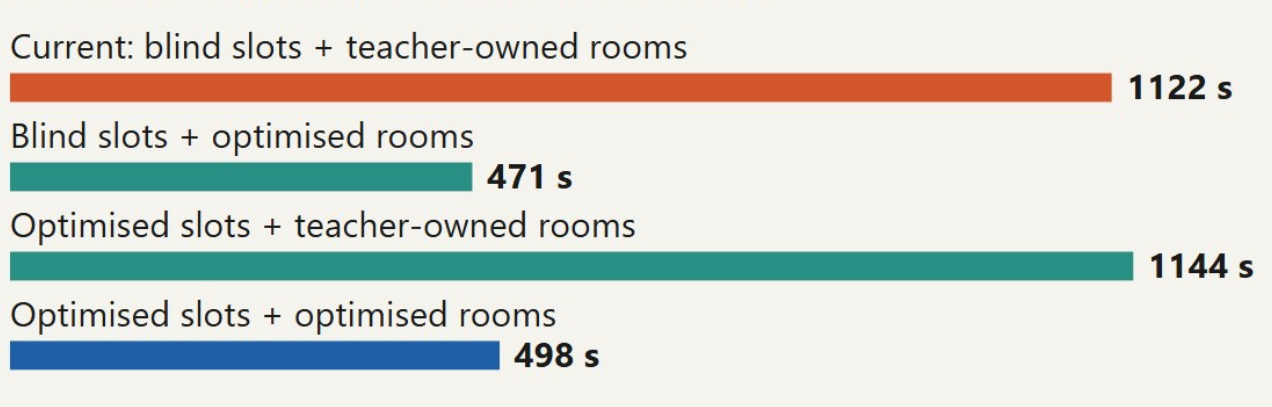


Fig. 14 — *We tried each change alone. Choosing rooms smartly removes the walking; choosing times smartly removes the free-period gaps.*

Every teacher's week

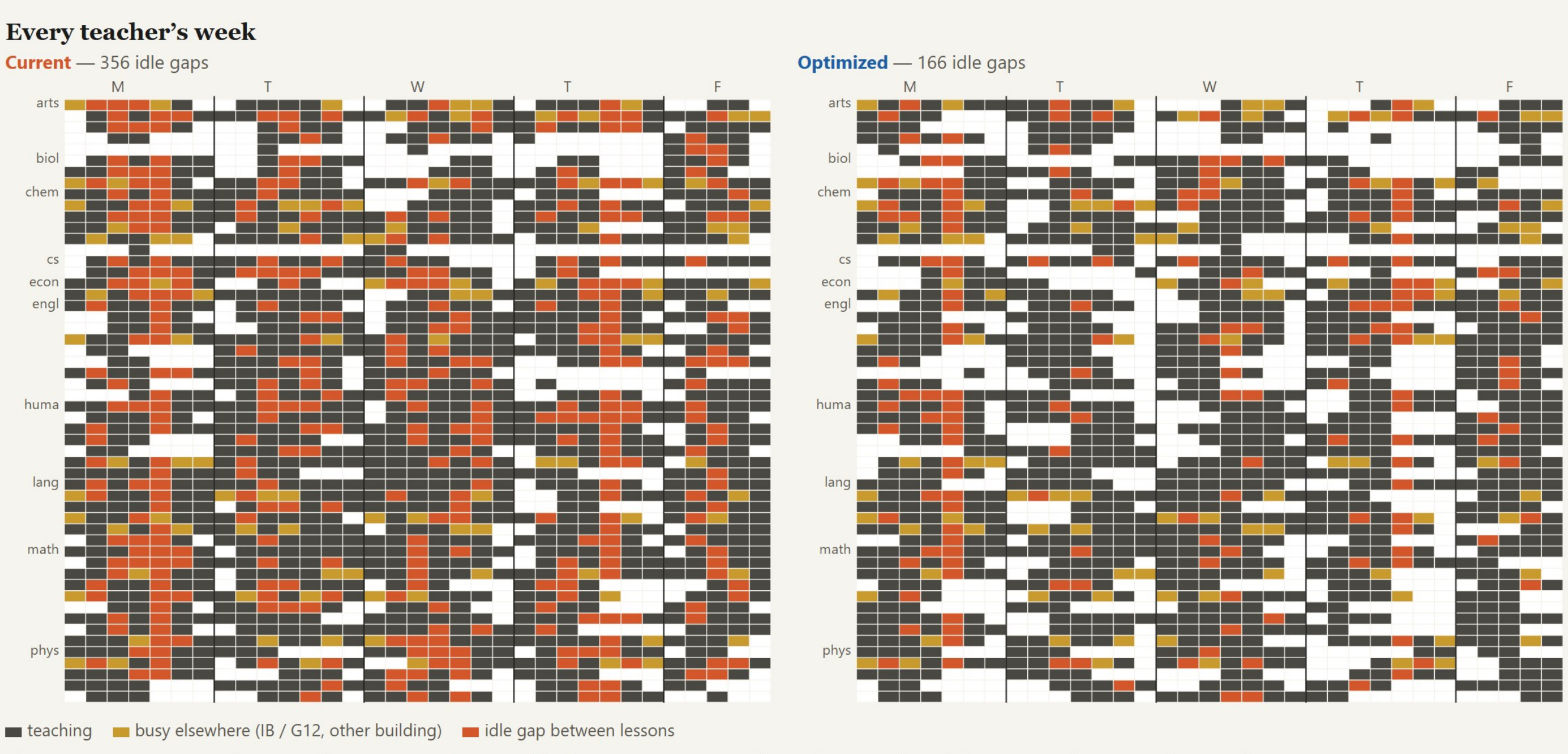


Fig. 15 — *One row per teacher, one column per period, grouped by department. Orange cells are free periods stuck between two lessons on the same day.*

Who benefits, and by how much

WHO	MEASURE (PER WEEK)	CURRENT	OPTIMIZED
626 students	walking (s)	1122	**498**
	room changes	19.9	**9.0**
	idle periods	2.77	**0.20**
	worst-20% walking (s)	1301	**724**
	hard subjects: mean period	3.74	**3.27**
54 teachers	idle gaps	6.7	**3.1**
	walking (s)	375	**439**
	schedule loss L	10.8	**10.3**
Everyone	sections off the minute band	110	**0**

How to read this

- Orange is the current method, blue is ours, on the same school and the same lessons.
- Students gain on every line.
- Teachers walk about 17% more. That is deliberate: it is cheaper for 54 teachers to carry the move than for 626 students.
- Teachers also lose about half their stuck free periods.

Students walk about half as much and almost never wait in a gap; teachers walk a bit more on purpose.

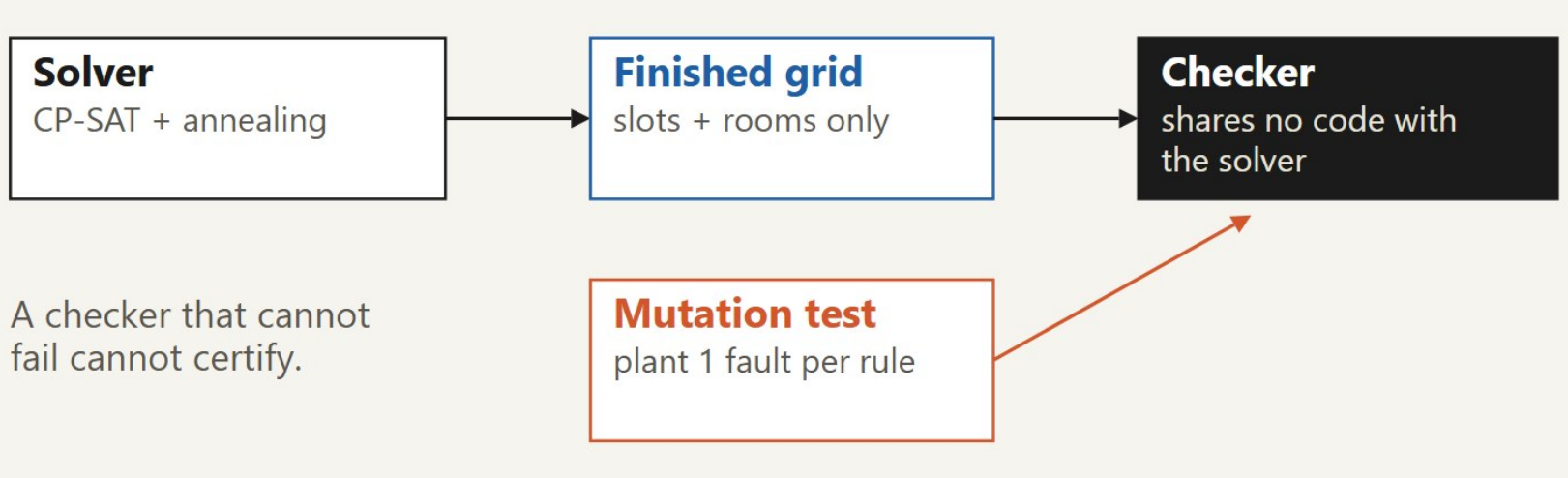
E How we know the rules are followed

Question 5

A second program, which shares no code with the scheduler, reads the finished timetable and checks ten rules. To be sure the checker can really catch mistakes, we deliberately break the timetable in ten different ways and confirm that it notices each one.

RULE	HARD CONSTRAINT	OURS	CURRENT	PLANTED FAULT
H1	section meets exactly its weekly slot count, on distinct slots	0	0	caught
H2	slot exists, is not a fixed block for the grade, not teacher-external-busy	0	0	caught
H3	no student in two places at once	0	0	caught
H4	no teacher double-booked; teacher qualified for the department	0	0	caught
H5	room exists, right kind, big enough, not double-booked	0	0	caught
H6	teacher consecutive-period cap	0	0	caught
H7	lab sections have a double period	0	0	caught
H8	weekly minutes within tolerance of nominal (contact-hours invariant)	0	110	caught
H9	each student receives every course's stated weekly slots	0	0	caught
H10	every consecutive-period room change reachable inside the break	0	0	caught

Fig. 16 — *270 sections, 626 students, 54 teachers, 723 room legs checked. Current = average over 24 simulated timetables.*



Solver CP-SAT + annealing

Finished grid slots + rooms only

Checker shares no code with the solver

A checker that cannot fail cannot certify.

Mutation test plant 1 fault per rule

Fig. 17 — *The checker works only from the finished timetable. The mistake test shows it can fail: all 10 deliberate mistakes were caught.*

The ten rules in plain words

- Every class gets all its lessons, in different periods.
- Lessons avoid PE time and the teacher's other commitments.
- No student, teacher or room is in two places at once.
- The room is the right kind and big enough; the teacher is qualified.
- Teachers never teach more in a row than their limit; labs get a double period.
- Weekly minutes stay near the normal amount.
- Every room change fits in the break between lessons.

20 automated tests also cover loads, course rules, the solver, repairs and the real week.

F What if our assumptions are wrong?

Question 6

Many numbers in a simulation are assumptions, so we changed each one, first one at a time and then several together, and re-ran everything. The gain stays in a narrow band in almost every case. The exceptions are real limits: a much heavier timetable, or losing a lot of rooms, can leave no valid timetable at all.

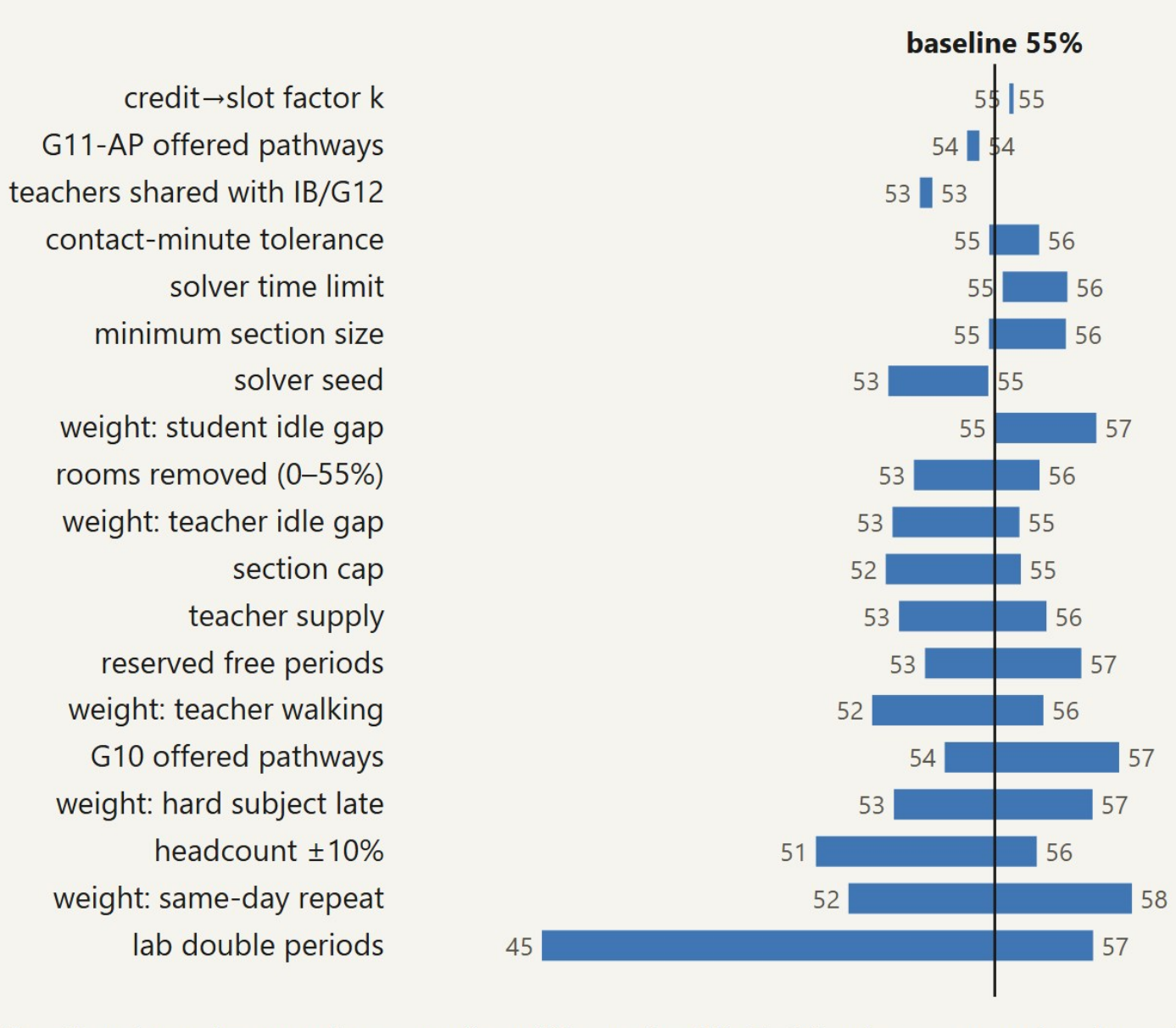


Fig. 18 — *How far each assumption moves the walking saving (%). Each bar is one assumption changed on its own; 61 runs, each solved and checked again (56 with no broken rules).*

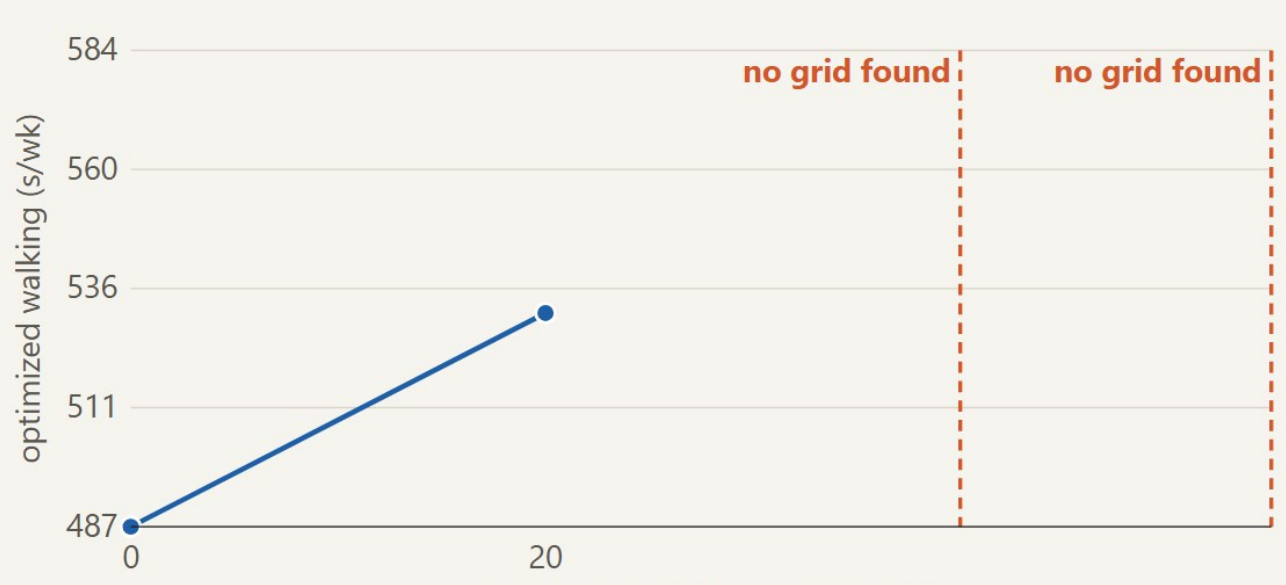


Fig. 19 — *Losing rooms: between 20% and 40% of rooms removed, no valid timetable exists. The program says so instead of giving a bad one.*

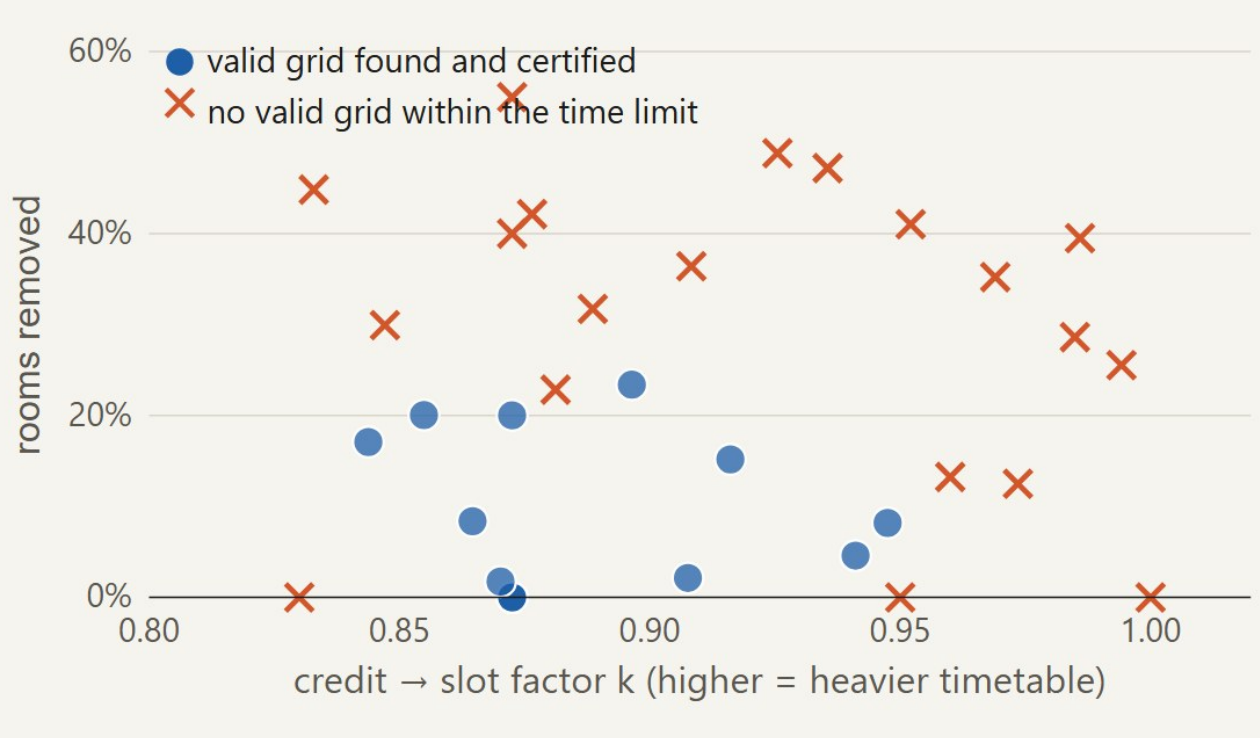


Fig. 20 — *Where it works. Each dot is one run. How heavy the timetable is (k) and how many rooms are lost decide whether a valid timetable can be found at all.*

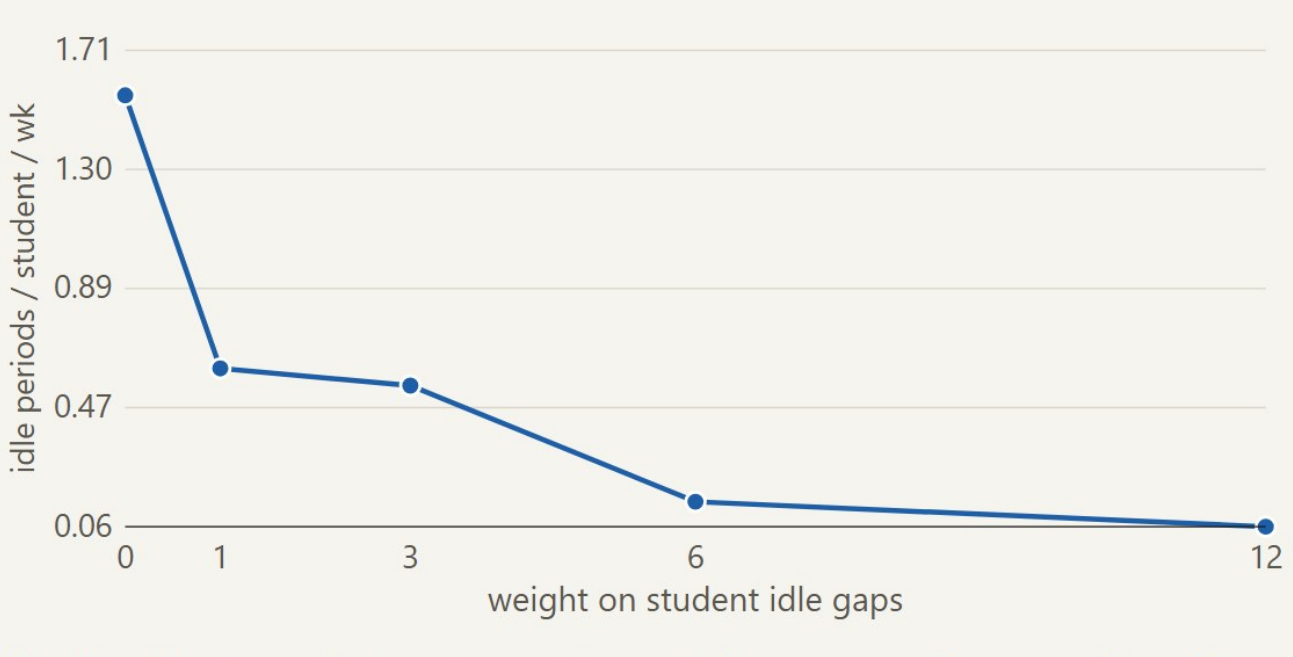


Fig. 21 — *Turning up the importance of free-period gaps removes them smoothly, with no sudden jump.*

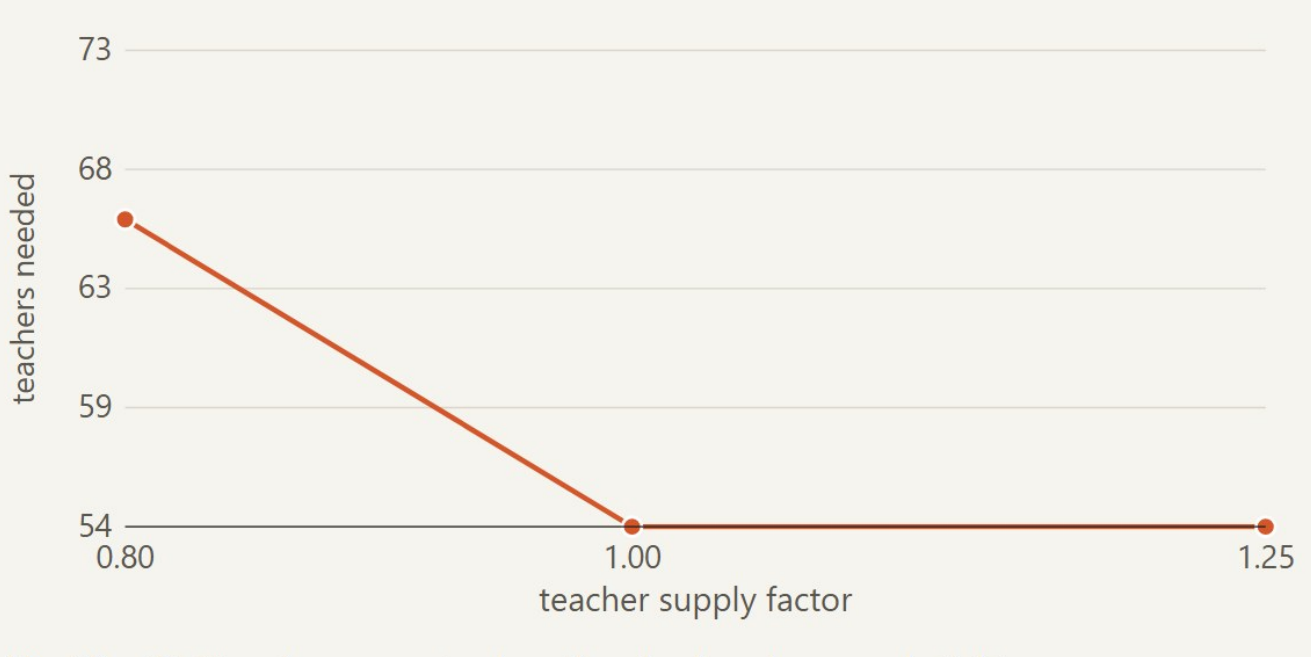


Fig. 22 — *Staffing: how many teachers the school needs as supply tightens.*

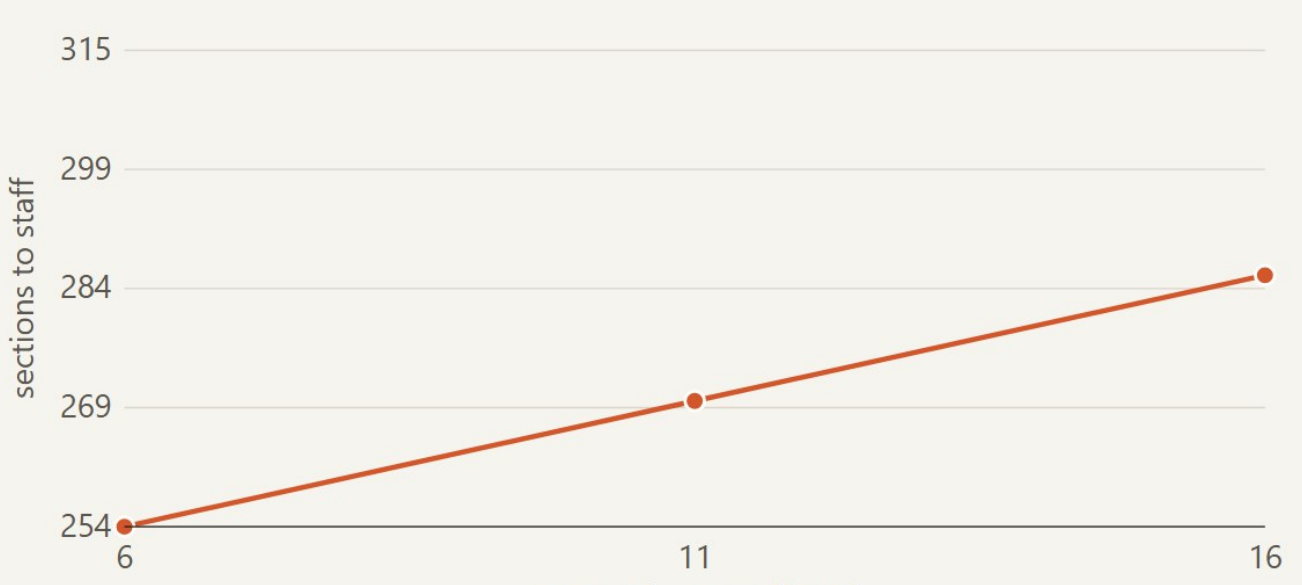


Fig. 23 — *The price of choice: offering more course packages means smaller classes and more classes to staff.*

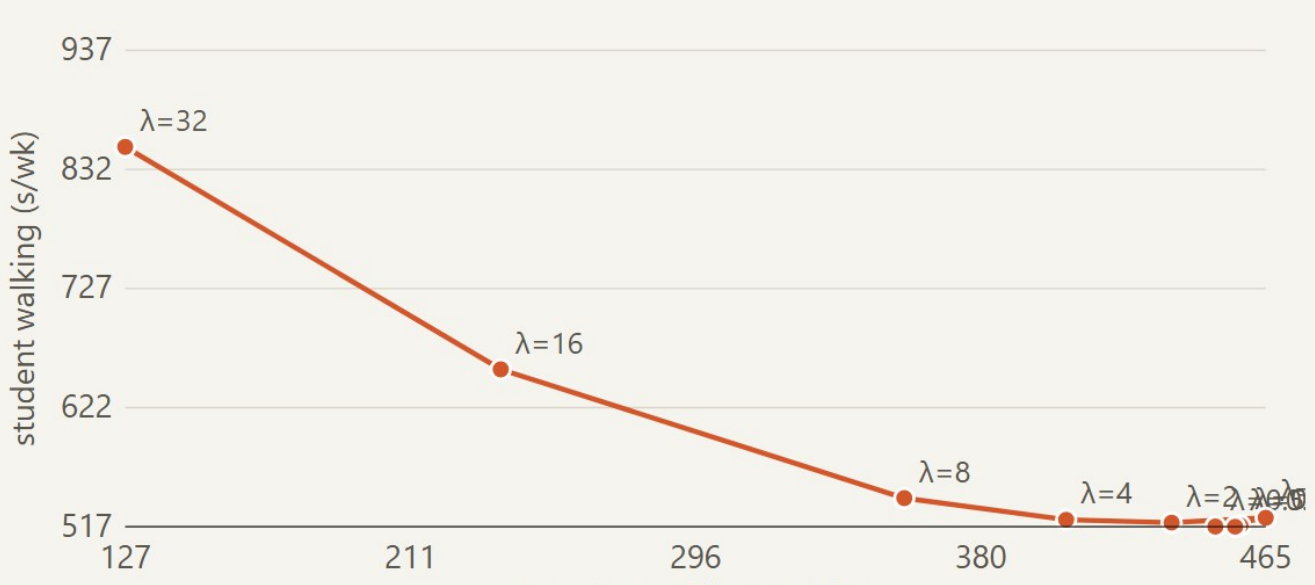


Fig. 24 — *Who walks: shifting walking from many students to few teachers is cheap, until a teacher second is weighted very heavily.*

What the sweeps tell us

- Across every single-assumption change, the walking saving stays between **45%** and **58%**.
- The gain holds up against the importance weights, class size limits, staffing and the solver random seed.
- It does **not** hold up against the load: a heavier timetable (k near 1) or losing 40% of rooms leaves no valid timetable, for the current method as well as ours.
- Offering more course packages costs more classes to staff.

G What happens when something breaks?

Question 6

Rooms close, teachers fall ill and enrollment grows. When that happens the program changes as little as it can: first it looks for a substitute, then it moves a lesson to another room, and only as a last step does it change a lesson's time. Every repaired timetable is checked again.

Room B217 closed for the week (B2)

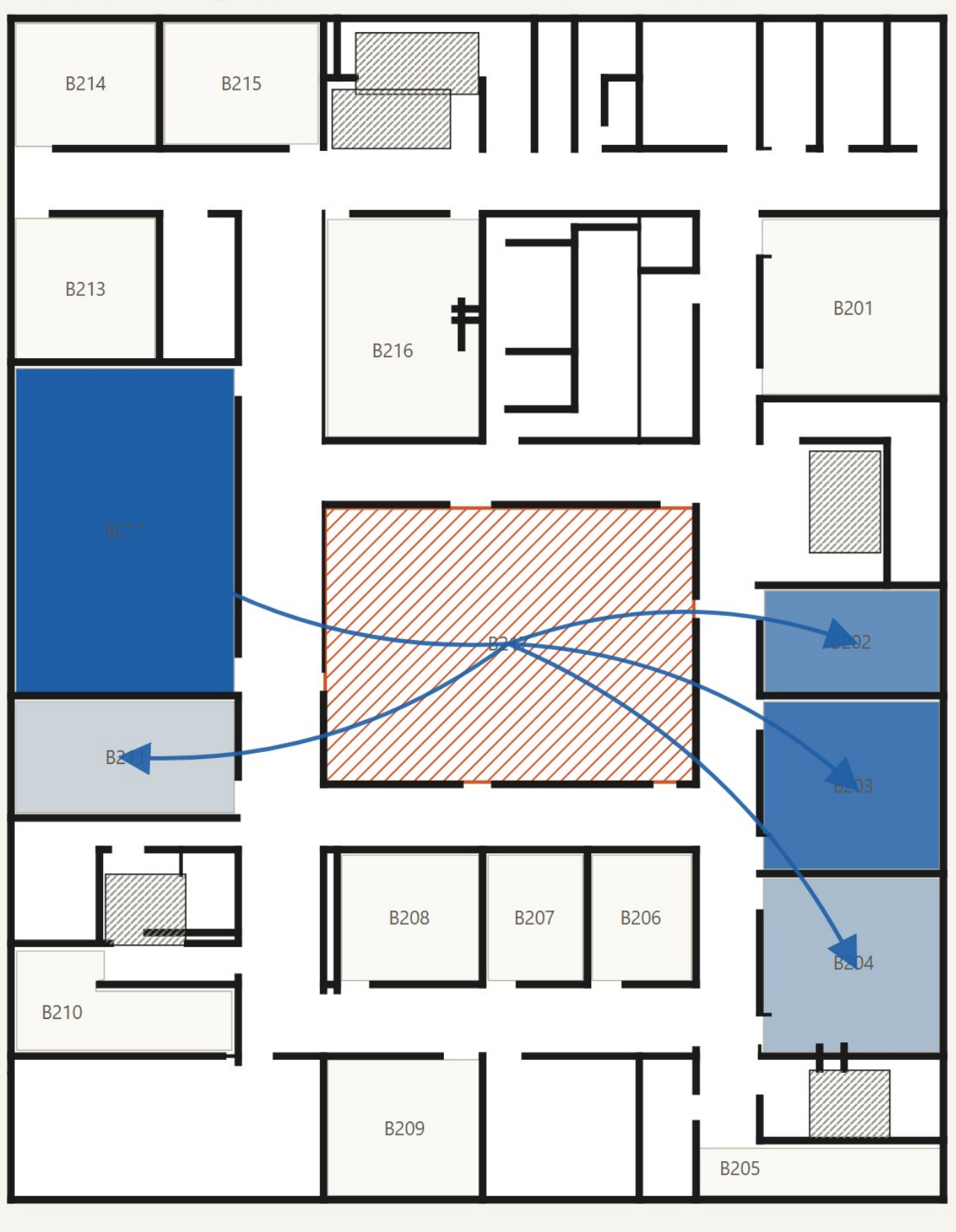


Fig. 25 — *28 lessons lose their room; 28 are re-roomed in a free room of the same kind,* **0 lessons change time**. *Blue = receiving rooms.*

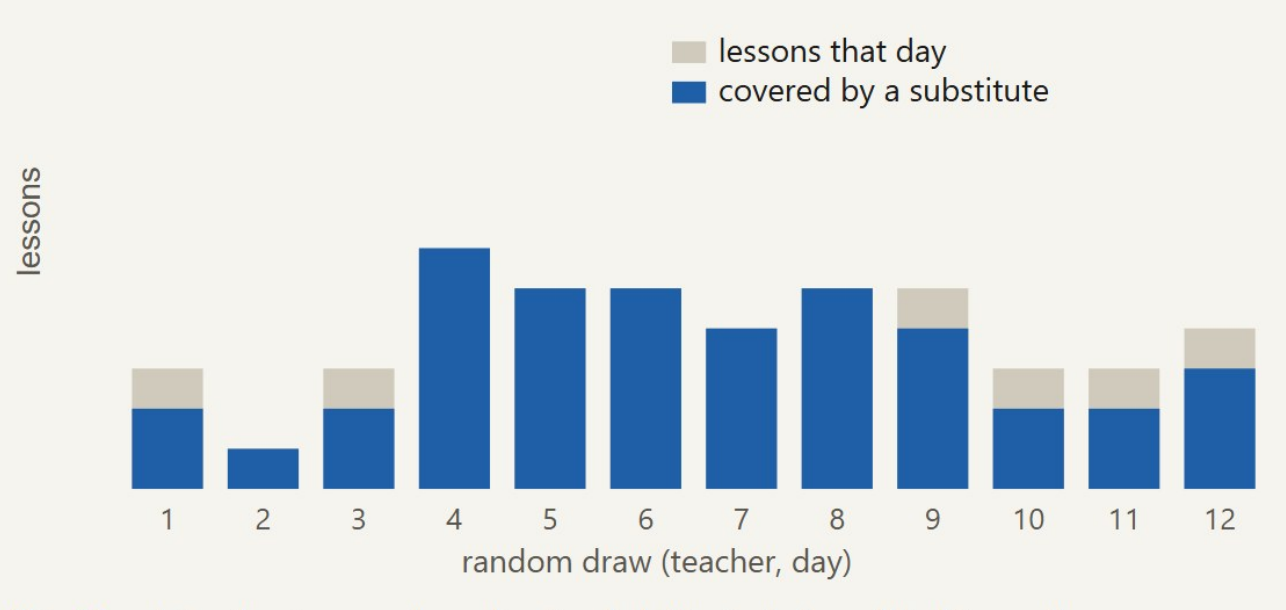


Fig. 26 — *A teacher is away for a day (probability 0.4). A qualified free colleague covers 87% of lessons and nothing in the timetable moves.*

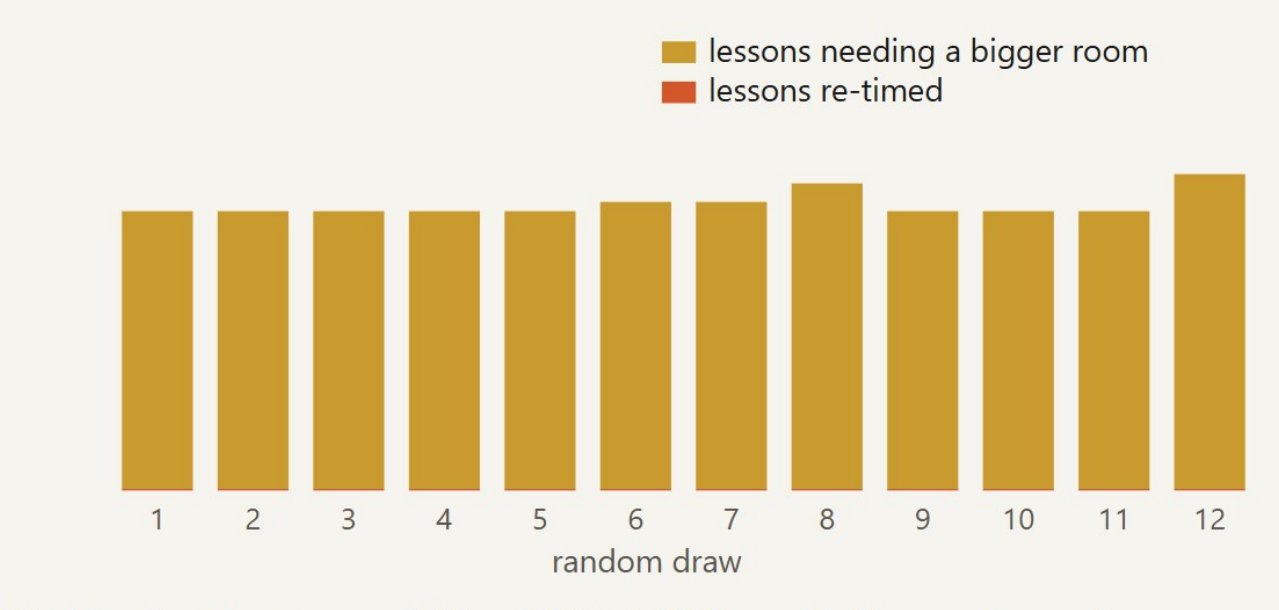


Fig. 27 — *Enrollment grows 5% (probability 0.3). Rooms that become too small are swapped for bigger ones; lesson times stay the same.*

The repair ladder

- **1. Substitute.** A qualified teacher who is free at that moment takes the lesson.
- **2. Move the room.** The lesson keeps its time and moves to a free room of the right kind and size.
- **3. Move the time.** Only if no room is free: just the affected classes are re-planned, everything else stays frozen.

Over 12 random draws of each event, no repair was impossible and no lesson had to change its time. Expected cost: **0.00** lessons moved and **14.4** room changes per event.

A closed room, a sick teacher or a growing intake is handled by small local changes, not a new timetable.

H What we have not proven, and how to re-run it

limits

- **One real week only.** The current system is an emulator calibrated to it (about 20 vs 21 changes); everything else is simulated. Registrar CSVs drop in.
- **Operating envelope.** Valid timetables were found at k ≈ 0.87 with up to 20% of rooms removed; at k ≥ 0.95 or ≥ 40% rooms removed none was found in the time limit (Fig. 20).
- **Not proven optimal.** CP-SAT stopped at its time limit, 78% from its bound: "optimized" means best feasible found.
- **Pathway classes trade choice for feasibility** (Fig. 4); the price is quantified in F.
- **Minutes:** sections of one course can still differ by up to 36% in weekly minutes; the next rule to add is equality across sections.
- **Reproduce:** `python -m tt.run`, `python -m tt.sensitivity`, `python -m tt.poster2`.
- Every number on this poster is read from `output/*.json`; seeds and the config hash are fixed.

Constraints from the course catalog · building: 6 floors, 103 rooms, stair-aware walking times · simulated school, seed 20261024

Python, OR-Tools CP-SAT